

Guía de Flujo del Proyecto: FakeNews Insight

1. Objetivo de esta guía

Esta guía te ayuda a entender cómo está construido el proyecto, cómo fluyen los datos entre frontend y backend, y qué archivos debes leer en orden para dominar el código sin perderte.

2. Visión general de la arquitectura

El proyecto está separado en dos aplicaciones:

- Frontend: `fakenews-frontend` (React + Vite + Zustand + Supabase JS)
- Backend: `fakenews-backend` (FastAPI + modelo SVM + Supabase REST)

Flujo principal:

1. El usuario interactúa en la UI (páginas/componentes).
2. La UI llama acciones en los stores de Zustand.
3. Los stores llaman servicios (`src/services/\*`).
4. Los servicios llaman a Supabase o al backend FastAPI.
5. El backend valida JWT, aplica reglas de negocio y persiste en Supabase.
6. La respuesta vuelve al store y de ahí a la UI.

3. Estructura de carpetas (resumen útil)

Frontend (`fakenews-frontend/src`)

- `App.jsx`: routing y guardas por sesión/rol.
- `main.jsx`: punto de entrada React.
- `pages/`: páginas completas (Landing, Login, Dashboard, Admin...).
- `components/`: piezas reutilizables de UI.
- `store/`: estado global (auth, análisis, dashboard, admin).
- `services/`: acceso a Supabase y backend.
- `lib/`: utilidades y reglas de acceso.

Backend (`fakenews-backend`)

- `main.py`: API FastAPI completa (auth, cuota, análisis, historial).

4. Flujo de autenticación y sesión

Archivos clave:

- `fakenews-frontend/src/App.jsx`
- `fakenews-frontend/src/store/authStore.js`
- `fakenews-frontend/src/services/auth.js`

Qué pasa al arrancar la app

1. ``App.jsx`` llama ``initializeAuth()``.
2. ``authStore`` obtiene sesión actual con Supabase (``getCurrentUser``).
3. Si hay usuario, hidrata perfil con ``getProfileByUserId``.
4. Se evalúa inactividad (timeout 1 hora).
5. ``App.jsx`` aplica rutas privadas o públicas según sesión y rol.

Gestión de inactividad

- ``App.jsx`` escucha eventos de actividad (``mousemove``, ``keydown``, etc.).
- Usa ``sessionActivity`` para guardar marca temporal por usuario.
- Si se supera el límite, fuerza ``logout()`` y limpia estado.

5. Flujo de navegación por roles

Archivo clave:

- ``fakenews-frontend/src/App.jsx``

Reglas activas:

- Usuario no autenticado: rutas públicas (Landing, Login, Register, Forgot Password).
- Usuario autenticado normal: ``Dashboard``.
- Usuario autenticado admin: redirige a ``AdminPanel``.
- Rutas admin (``/admin``, ``/admin/users``) protegidas por rol.

6. Flujo de análisis (core funcional)

Archivos clave:

- ``fakenews-frontend/src/pages/Dashboard.jsx``
- ``fakenews-frontend/src/store/analysisStore.js``
- ``fakenews-frontend/src/services/analysis.js``
- ``fakenews-backend/main.py``

Paso a paso

1. En Dashboard se envía texto para analizar.
2. ``analysisStore.analyzeText()`` obtiene JWT con ``getAccessToken()``.
3. ``services/analysis.analyzeTextNews()`` hace POST a ``/predecir/``.
4. Backend valida JWT contra Supabase (``/auth/v1/user``).
5. Backend carga perfil y aplica cuota diaria (plan free) o ilimitado (pro/ultra).
6. Backend limpia texto con NLTK y ejecuta modelo SVM + TF-IDF.
7. Backend guarda ejecución en ``analysis_runs``.

8. Devuelve veredicto + metadatos (``analysis_run_id``, confianza, etc.).
9. Store transforma respuesta al formato de UI y renderiza el resultado.

7. Flujo de guardado en historial

Archivos clave:

- ``fakenews-frontend/src/store/analysisStore.js``
- ``fakenews-frontend/src/services/analysis.js``
- ``fakenews-backend/main.py``

Paso a paso

1. Desde resultado, UI ejecuta guardar manualmente.
2. Store llama ``saveAnalysisToHistory({ runId, jwtToken })``.
3. Backend endpoint ``/analyses/save`` valida ownership del ``run_id``.
4. Inserta en ``analyses`` (historial persistente).
5. Marca ``analysis_runs.saved_to_history = true``.
6. Frontend marca ``savedInHistory`` para evitar duplicados.

8. Flujo de Home del Dashboard

Archivos clave:

- ``fakenews-frontend/src/store/dashboardStore.js``
- ``fakenews-frontend/src/services/dashboard.js``

Qué calcula

- Métricas de uso diario (``usedToday``, ``remainingLabel``, ``dailyLimitLabel``).
- Actividad mensual (``usedThisMonth``).
- Serie de últimos 10 días para gráfica.
- Últimos análisis normalizados para tarjetas recientes.

``services/dashboard.js`` intenta leer distintas tablas posibles (``analysis_runs``, ``analysis_history``, ``analysis_results``, ``analyses``) para ser robusto ante cambios de esquema.

9. Flujo del panel admin

Archivos clave:

- ``fakenews-frontend/src/pages/AdminPanel.jsx``
- ``fakenews-frontend/src/pages/AdminUsers.jsx``
- ``fakenews-frontend/src/store/adminStore.js``
- ``fakenews-frontend/src/services/admin.js``

Funciones principales

- Carga usuarios paginados.
- Filtro por nombre o UUID.
- KPIs por plan.
- Cambio de plan y baja de usuario.

Nota de estado actual:

- El popup de planes en dashboard se mantiene como UI.
- El flujo avanzado de pago/cambio de plan quedó pausado para retomarlo más adelante.

10. Backend: reglas de negocio importantes

Archivo clave:

- ``fakenews-backend/main.py``

Reglas críticas

- Validación JWT obligatoria para endpoints privados.
- Cuota diaria para plan free con control de concurrencia.
- Planes pro/ultra marcados como ilimitados.
- Persistencia separada entre:

- ``analysis_runs``: ejecución técnica.

- ``analyses``: historial guardado por usuario.

11. Orden recomendado para estudiar el código

Si quieres entender todo rápido, sigue este orden:

1. ``fakenews-frontend/src/main.jsx``
2. ``fakenews-frontend/src/App.jsx``
3. ``fakenews-frontend/src/store/authStore.js``
4. ``fakenews-frontend/src/services/auth.js``
5. ``fakenews-frontend/src/pages/Dashboard.jsx``
6. ``fakenews-frontend/src/store/analysisStore.js``
7. ``fakenews-frontend/src/services/analysis.js``
8. ``fakenews-frontend/src/store/dashboardStore.js``
9. ``fakenews-frontend/src/services/dashboard.js``
10. ``fakenews-backend/main.py``
11. ``fakenews-frontend/src/pages/AdminPanel.jsx``
12. ``fakenews-frontend/src/pages/AdminUsers.jsx``
13. ``fakenews-frontend/src/store/adminStore.js``
14. ``fakenews-frontend/src/services/admin.js``

12. Checklist rápida para onboarding

- Levantar frontend y backend localmente.
- Probar login/register/logout.
- Probar análisis de texto y guardar historial.
- Revisar logs de backend al consumir cuota free.
- Entrar a panel admin y validar carga de usuarios.
- Leer stores para entender dónde vive cada estado.

13. Mapa mental final (resumen)

- UI decide qué mostrar.
- Store decide cómo orquestar.
- Service decide cómo hablar con backend/Supabase.
- Backend decide reglas de negocio y persistencia.

Si mantienes esta separación al programar cambios, el proyecto seguirá estable y fácil de mantener.